

บทที่ 4

การจัดการกับวินโดวส์

การจัดการกับวินโดวส์

4.1 เมสเซจ

เมสเซจเป็นอินพุตหรือข้อมูลเข้าเพียงทางเดียวของแอปพลิเคชัน วินโดวส์เป็นตัวแทนต่อทุก ๆ เหตุการณ์ทั้งหมดที่ต้องการตอบสนอง เมสเซจเป็นตัวแปร โครงสร้างชนิดหนึ่งที่ประกอบไปด้วยส่วนค่าอ้างอิงของเมสเซจนั้น กับส่วนพารามิเตอร์ โดยที่พารามิเตอร์จะขึ้นกับเมสเซจชนิดนั้น ๆ

4.1.1 การสร้างและการประมวลผลเมสเซจ

วินโดวส์จะมีการสร้างเมสเซจขึ้นทุกครั้งที่มีอินพุตเข้ามายังระบบไม่ว่าจะเป็นการเลื่อนเมาส์ การกดคีย์ หรืออื่น ๆ เพื่อบอกแก่แอปพลิเคชันหรือตัววินโดวส์เองถึงเหตุการณ์ (อินพุต) ที่เกิดขึ้น โดยวินโดวส์จะนำเมสเซจที่สร้างขึ้นเหล่านี้ใส่เข้าไปในคิวของระบบจากนั้นก็ส่งผ่านไปยังคิวของแอปพลิเคชันที่เหมาะสมต่อไป ลักษณะของคิวของแอปพลิเคชันนี้เป็นแบบเข้าก่อนออกก่อน โดยที่การเข้าก็คือวินโดวส์ดึงเมสเซจจากคิวของระบบมาใส่ให้แก่คิวของแอปพลิเคชันส่วนการออกคือแอปพลิเคชันเป็นตัวดึงออกไปเอง โดยเรียกใช้ฟังก์ชัน GetMessage จากนั้นก็แจกแจงและส่งไปยังฟังก์ชันประจำวินโดวส์ต่าง ๆ ด้วยฟังก์ชัน DispatchMessage

มีเมสเซจบางอันที่วินโดวส์ต้องการส่งไปยังฟังก์ชันประจำวินโดวส์โดยตรง แทนที่จะวางลงในคิว อาจเรียกเมสเซจนี้ว่าเป็น “เมสเซจลัดคิว” เมสเซจลัดคิวนี้จะเป็นเมสเซจที่มีผลกับวินโดวส์นั้น ๆ โดยเฉพาะวินโดวส์และแอปพลิเคชันประจำวินโดวส์โดยตรงอีกทั้งฟังก์ชันประจำวินโดวส์นั้นยังสามารถส่งค่าการประมวลผลเมสเซจกลับมาได้อีกด้วย

ตัวอย่างของเมสเซจลัดคิวอย่างหนึ่งก็คือ การใช้ฟังก์ชัน CreateWindow นอกจากจะเป็นการสร้างวินโดวส์แล้ว ยังเป็นการบังคับให้วินโดวส์ส่งเมสเซจ WM_CREATE ซึ่งเป็นเมสเซจลัดคิวไปยังฟังก์ชันประจำวินโดวส์ แล้วรอนกว่าฟังก์ชันนั้นจะจัดการกับเมสเซจเสร็จ เมสเซจ WM_CREATE จะไม่ผ่านแอปพลิเคชันเลย

แต่ก็ไม่ใช่ว่ามีเพียงวินโดวส์เท่านั้นที่สามารถสร้างเมสเซจขึ้นมาได้ตัวแอปพลิเคชันเองก็สามารถสร้างเมสเซจแล้วส่งไปยังคิวของตนเอง และคิวของแอปพลิเคชันอื่น ๆ ได้เช่นกัน

แอปพลิเคชันจะใช้ฟังก์ชัน GetMessage ในลูปฟังก์ชัน WinMain เพื่อดึงเมสเซจออกจากคิวของตัวเอง แอปพลิเคชัน ฟังก์ชัน GetMessage จะทำงานโดยเริ่มจากดูว่าในคิวมีเมสเซจหรือไม่หากมีก็จะดึงเมสเซจที่อยู่แรกสุดไป แต่หากไม่มีเมสเซจอยู่ในคิว ก็จะมีการรอเมสเซจขึ้น และปล่อยการควบคุมกลับคืนไปให้วินโดวส์ เพื่อแบ่งการควบคุมแอปพลิเคชันอื่นสามารถทำงานได้ต่อไป

เมื่อฟังก์ชัน WinMain ของแอปพลิเคชันได้รับเมสเสจจากคิวแล้วก็ต้องใช้เรียกฟังก์ชัน DispatchMessage เพื่อแจกแจงเมสเสจและการส่งเมสเซนนั้นไปให้ฟังก์ชันประจำวัน ไควส์ต่างๆการทำเช่นนี้จะเป็นการถ่ายการควบคุมให้กับฟังก์ชันประจำวัน ไควส์ที่จะประมวลผลเมสเซนนั้นหลังจากที่ทำงานเกี่ยวข้องกับเมสเซนนี้เรียบร้อยแล้ว ก็ถ่ายการควบคุมกลับไปให้ฟังก์ชันหลักในแอปพลิเคชันเพื่อคิงเมสเซนต่อไปในคิวมาประมวลผลต่อไป

ข้อควรระวัง ฟังก์ชันประจำวัน ไควส์ไม่ควรคาดว่าเมสเซนจะเข้ามาก่อน เพราะวิน ไควส์สามารถส่งเมสเซนมาในลำดับอย่างไรก็ได้

ส่วนการรับข้อมูลจากคีย์บอร์ดนั้นวิน ไควส์จะรับเข้ามาในลักษณะของรหัสของคีย์ที่กด ดังนั้นจึงต้องมีการแปลงรหัสของคีย์เหล่านั้นให้เป็นรหัสของตัวอักษรเมื่อแปลงได้แล้วฟังก์ชันก็จะส่งเมสเซนของตัวเองเข้าไปในคิว พร้อมทั้งรหัสตัวอักษรที่ผู้ใช้กดเข้ามาเป็นพารามิเตอร์ของเมสเซนนั้น

4.1.2 การแปลและตีความเมสเซน

โดยทั่วไปแล้วจะมีการใช้ฟังก์ชัน TranslateMessage นี้กับทุกๆเมสเซนที่เข้ามา โดยที่หากไม่ใช่เมสเซนที่เกี่ยวกับคีย์บอร์ดก็จะไม่มีผลอันใด

ตัวอย่างต่อไปนี้จะเป็นการแสดงถึงหน้าต่างของเมสเซนดูอยู่ที่ฟังก์ชัน WinMain ของแอปพลิเคชัน เมสเซนดูจะคิงเมสเซนออกจากคิวและแจกจ่ายดังนี้

```
int PASCAL WinMain ( hInst, hPrevInst, lpCmdLine, ShowCmd )
HINSTANCE hInst ;
HINSTANCE hPrevInst ;
LPSTR lpCmdLine ;
int ShowCmd ;
{
    MSG msg ;

    .
    .
    .

    While ( GetMessage ( &msg, NULL, 0,0 ) ) {
        TranslateMessage ( &msg ) ;
        DispatchMessage ( &msg ) ;
    }

    return msg.wParam;
}
```

```
}
```

ส่วนแอปพลิเคชันที่มีการใช้คีย์ทันใจ (accelerator key) ของเมนู ก็ต้องมีการสร้างตารางคีย์ทันใจที่ใช้ไว้ในไฟล์รีซอร์สจากนั้นก็ต้องมีการโหลดตารางนี้เข้ามาโดยใช้ฟังก์ชัน LoadAccelerator โดยจะทำให้ลักษณะของรูปในการรับส่งเมสเสจเป็นดังนี้

```
While ( GetMessage ( ( ( LPMSG ) &msg, ( HWND ) NULL, 0, 0 ) ) )
{
    if ( TranslateAccelerator ( hWnd, hAccel, ( ( LPMSG ) &msg ) == 0 )
    {
        TranslateMessage ( ( LPMSG ) &msg );
        DispatchMessage ( ( LPMSG ) &msg );
    }
}
```

ที่ต้องการตรวจสอบว่าค่าที่ได้กลับมาจากการเรียกใช้ฟังก์ชัน TranslateAccelerator นั้นก็เพราะว่าจากเมสเซนจ์นั้นเป็นการกดคีย์ทันใจ ฟังก์ชัน TranslateAccelerator ก็จะทำการแปลงและส่งเมสเสจที่แปลงแล้วนั้น ไปยังฟังก์ชันประจำวินโดวส์เอง ดังนั้นจึงไม่ต้องมาเรียกใช้ฟังก์ชัน TranslateMessage และฟังก์ชัน DispatchMessage อีก

4.1.3 การตรวจสอบเมสเสจ

เราใช้ฟังก์ชัน PeekMessage เพื่อมองหาและตรวจสอบเมสเสจบางเมสเสจในระหว่างการทำงานที่กินเวลา โดยไม่ต้องการมีผลกระทบกระเทือนต่อการเป็นอยู่ของเมสเสจ เมื่อเรียกใช้ฟังก์ชันนี้ ถ้ามีเมสเสจในคิวฟังก์ชันนี้จะให้ค่าที่ไม่เป็น 0 กลับมา ทำให้สามารถประมวลเมสเสจได้โดยไม่ต้องกลับไปทีลูปของฟังก์ชัน WinMain ประโยชน์ของฟังก์ชันนี้คือ เมื่อมีการทำงานบางอย่างที่ต้องกินเวลามาก (ใช้เวลานานกว่าจะกินการควบคุมไปให้ฟังก์ชันหลัก) เช่น การอ่านเขียนดิสก์ ก็สามารถให้ฟังก์ชันนี้ในการตรวจสอบว่า ขณะนี้ผู้ใช้มีการกดคีย์เพื่อยกเลิกการทำงานหรือไม่

4.1.3 การส่งและส่งผ่านเมสเสจ

ฟังก์ชัน Postmessage และ SendMessage มีหน้าที่ในการส่งเมสเสจไปยังฟังก์ชันประจำวินโดวส์ของตัวเองหรือฟังก์ชันประจำวินโดวส์ของแอปพลิเคชันอื่น นอกจากนี้ยังมีฟังก์ชัน PostAppMessage ที่ทำงานแบบเดียวกับฟังก์ชัน PostMessage แต่เป็นการส่งผ่านโดยอาศัยแฮนเดิลโมดูลของแอปพลิเคชันแทน

ฟังก์ชัน PostMessage จะส่งเมสเสจผ่านไปทางคิวของแอปพลิเคชัน (เราจะขอเรียกการส่งแบบนี้ว่าการส่งผ่าน) โดยที่ผู้ส่งจะยังไม่เสียการควบคุม และผลของเมสเสจที่ส่งไปนั้นก็จะยังไม่เกิดขึ้นจนกว่าแอปพลิเคชันนั้น ๆ จะดึงเมสเสจที่ส่งไปนั้นขึ้นมา ส่วน SendMessage จะส่งเมสเสจลัดคิวไปยังฟังก์ชันประจำวันโควล์ที่ต้องการทันที (โดยเราจะขอเรียกการส่งแบบนี้ว่าการส่ง) โดยที่ไม่ต้องผ่านคิวของแอปพลิเคชัน ซึ่งการส่งเมสเสจไปแบบนี้ ผู้ที่ส่งจะสูญเสียการควบคุมไปให้ฟังก์ชันประจำวันโควล์ที่ส่งเมสเสจให้ และเมื่อฟังก์ชันทำงานเสร็จแล้วก็จะคืนการควบคุมกลับมา ส่วนค่าที่ได้กลับมาของการเรียกใช้ SendMessage นั้นคือค่าที่ฟังก์ชันประจำวันโควล์นั้นส่งค่ากลับมา แต่ถ้าเป็นค่าที่ได้รับกลับมาจากฟังก์ชัน PostMessage เป็นเพียงว่าสามารถส่งเมสเสจนั้นไปได้สำเร็จหรือไม่

4.1.4 ฟังก์ชันที่ใช้กับเมสเสจ

ฟังก์ชันในกลุ่มนี้มีหน้าที่ในการอ่านเมสเสจขึ้นมาจากคิวและประมวลผลเมสเสจในวินโดวส์ ฟังก์ชันที่ใช้กับเมสเสจก็จะมีตารางดังนี้

ฟังก์ชัน	หน้าที่
CallWindowProc	ส่งผ่านข้อมูลของเมสเสจไปยังฟังก์ชัน
DispatchMessage	แจกแจงเมสเสจและส่งเมสเสจไปยังฟังก์ชันประจำวันโควส์ใด ๆ
GetMessage	รับเมสเสจในช่วงของเมสเสจที่กำหนดไว้
GetMessagePost	ให้ค่าตำแหน่งของเมาส์ในขณะที่มีการรับเมสเสจล่าสุด
GetMessageTime	ให้ค่าเวลาในขณะที่มีการรับเมสเสจล่าสุด
PeekMessage	ตรวจสอบว่ามีเมสเสจในคิวหรือไม่โดยไม่ได้ออกมา
PostAppMessage	ส่งเมสเสจไปยังแอปพลิเคชัน
PostMessage	ส่งเมสเสจไปยังคิวของแอปพลิเคชัน
PostQuitmessage	ส่ง WM_QUIT ไปยังแอปพลิเคชัน
ReplyMessage	ตอบรับเมสเสจ
SendMessage	ส่งเมสเสจไปยังวินโดวส์ต่าง ๆ
TranslateAccelerator	ประมวลผลคีย์ที่กดของเมนูให้เป็นเมสเสจและส่งเมสเสจให้แก่วินโดวส์
Translatemessage	แปลคีย์ของคีย์ที่กดให้เป็นรหัสตัวอักษร
WaitMessage	ส่งการควบคุมไปให้แอปพลิเคชันอื่น
Winmain	เป็นจุดที่เริ่มเข้าไปทำงานของวินโดวส์แอปพลิเคชัน

4.2 ช่องเกี่ยวโยง (Hook)

ช่องเกี่ยวโยงนับเป็นกลไกการจัดการเมสเสจอย่างหนึ่งของวินโดวส์ ที่สามารถให้แอปพลิเคชันสามารถเข้าถึงเมสเสจที่ส่งกันไปมาวินโดวส์มีช่องเกี่ยวโยงให้ใช้กันหลายแบบซึ่งแต่ละแบบก็จะมีความสามารถในการเข้าถึงเมสเสจต่างกันไป การใช้ประโยชน์จากช่องเกี่ยวโยงคือการติดตั้งฟังก์ชันกรองเมสเสจ (filter function) ซึ่งจะจัดการกับเมสเสจจากช่องเกี่ยวโยงก่อนที่เมสเสจนั้นจะจำวินโดวส์ที่อยู่ปลายทาง

ลูกโซ่ของฟังก์ชันการกรองเมสเสจ

ลูกโซ่ของฟังก์ชันการกรองเมสเสจเกิดจากการต่อฟังก์ชันการกรองเมสเสจหลายๆ ฟังก์ชันเข้ากับช่องเกี่ยวโยงเดียวกันของระบบ โดยที่ตำแหน่งแรกสุดของลูกโซ่คือ ช่องเกี่ยวโยงนั่นเอง และปลายของลูกโซ่ก็จะไปอยู่ที่ฟังก์ชันประจำวินโดวส์ เมื่อเราติดตั้งฟังก์ชันการกรองเมสเสจ ฟังก์ชันดังกล่าวจะเกี่ยวข้องกับช่องเกี่ยวโยงอันถือว่าเป็นจุดเริ่มต้นของลูกโซ่รับเมสเสจแล้วส่งเมสเสจที่กรองแล้วโยงเข้ากับฟังก์ชันการกรองเมสเสจที่เคยติดตั้งอยู่ ซึ่งตอนนี้อยู่ในตำแหน่งถัดไป ฟังก์ชันการกรองถัดไปนั้นก็จะทำเช่นนี้ต่อไปเรื่อย ๆ เป็นลูกโซ่ที่จะโยงเข้าหาฟังก์ชันประจำวินโดวส์ในที่สุด

ฟังก์ชันการกรองเมสเสจจะสามารถโยงตัวเองเข้ากับช่องเกี่ยวโยงด้วยฟังก์ชัน SetWindowHook ในแต่ละครั้งที่เรียก ฟังก์ชันการกรองเมสเสจเหล่านั้นก็จะถูกโยงไปยังจุดเริ่มต้นของลูกโซ่ (ซึ่งอาจมีฟังก์ชันการกรองเมสเสจเกี่ยวข้อง) นั่นคือโยงต่อเข้ากับช่องเกี่ยวโยง และทุกครั้งที่แอปพลิเคชันติดตั้งฟังก์ชันการกรองเมสเสจเข้ากับช่องเกี่ยวโยง ก็จะต้องสำรองพื้นที่สำหรับเก็บตำแหน่งแอดเดรสของฟังก์ชันการกรองเมสเสจที่เคยอยู่เดิม เพื่อส่งเมสเสจที่กรองแล้วโยงเข้ากับฟังก์ชันการกรองที่อยู่ถัดไป

การจะถอดฟังก์ชันการกรองเมสเสจออกจากลูกโซ่คือแอปพลิเคชันเรียกใช้ฟังก์ชัน UnHookWindowHook ด้วยชนิดของช่องเกี่ยวโยงและตัวชี้ไปยังฟังก์ชันการกรองนั้น

ช่องเกี่ยวโยงวินโดวส์และช่องเกี่ยวโยงคีย์บอร์ดมีตารางนี้

ชนิด	จุดประสงค์
WH_CALLWNDPROC	ติดตั้งฟังก์ชันการกรองเมสเสจของวินโดวส์
WH_CBT	ติดตั้งฟังก์ชันการกรองเมสเสจสำหรับการช่วยการสอน
WH_DEBUG	ติดตั้งฟังก์ชันการกรองเมสเสจสำหรับการดีบั๊ก
WH_GETMESSAGE	ติดตั้งฟังก์ชันการกรองเมสเสจ (เฉพาะวินโดวส์รุ่น คีบัก)
WH_HARDWARE	ติดตั้งฟังก์ชันการกรองเมสเสจสำหรับฮาร์ดแวร์ที่ไม่เป็นมาตรฐานของวินโดวส์
WH_JOURNALPLAYBACK	ติดตั้งฟังก์ชันการกรองเมสเสจเล่นกลับ
WH_JOURNALRECORD	ติดตั้งฟังก์ชันการกรองเมสเสจบันทึก
WH_KEYBOARD	ติดตั้งฟังก์ชันการกรองเมสเสจคีย์บอร์ด

WH_MOUSE	ติดตั้งฟังก์ชันกรองเมสเสจเมาส์
WH_MSGFILTER	ติดตั้งฟังก์ชันกรองเมสเสจทั่วไป
WH_SYSMSGFILTER	ติดตั้งฟังก์ชันกรองเมสเสจทั้งหมด

เพิ่มเติม ช่องกี่ยวโยระบบ WH_CALLWNDPROC และ WH_GETMESSAGE จะมีผลกระทบต่อประสิทธิภาพของระบบ จึงควรใช้เฉพาะในการดื่บักเท่านั้น